DERİN TARAMA RAPORU · MYGAMEAREA

LegendySF-V2 + Dump.rar Derin Analiz

Metin2 hareket protokolü, **SendCharacterStatePacket** ve adım atma animasyonu mekanizması — C++ kaynak kodu, Python dump katmanı ve topluluk kanıtlarıyla satır satır çözümleme.

KAPSAM

154 Python dosyası · ~835 C++ binding · FUNC_WAIT mekanizması

28 Ekim 2026 · Dosya-satır kanıtlı derin tarama

İçindekiler

1. Yönetici Özeti **2**

2. Kaynaklar ve Yöntem **3**

3. dump.rar Envanteri: Python Katmanı **4**

4. LegendySF-V2 Envanteri: C++ Katmanı **5**

5. C++ vs Python Karşılaştırma Matrisi **7**

6. Hareket Protokolü: Katman Katman **10**

7. Adım Atma Animasyonu Mekanizması (Çekirdek Bölüm) **12**

8. Bilinçli mi? Motivasyon ve Sunucu Tespiti **15**

9. Uygulama Notları, Sonuç ve Kaynakça **16**

1. Yönetici Özeti

Bu rapor, üç kaynağın uçtan uca statik analizinin ürünüdür: DrLegendy/LegendySF-V2 deposundaki gerçek Metin2 istemci ve sunucu C++ kaynak kodu, içine kendi pydumpall aracınızın (PLUSMAXGOD pydumpall v4.1) çalıştırıldığı bir istemciden toplanan dump.rar Python katmanı (156 dosya) ve MyGameArea çalışma alanında biriken önceki analiz mirası (DEV_Report.md, 1350 fonksiyonluk envanter). Raporun iki ana hedefi vardı: birincisi, dump.rar içindeki Python fonksiyonlarını LegendySF-V2'nin C++ yetenekleriyle satır satır eşleştirmek ve hangi C++ işinin Python'dan yapılabileceğini kesin sınırlarıyla ortaya koymak; ikincisi, sorunuzdaki bot davranışını — karakterin tek ayağı havada kalacak şekilde, adım atma animasyonu hiç tamamlanmadan yürümesi — paket protokolü ve animasyon motoru seviyesinde kök nedenine inerek çözmek. Her iki hedef de dosya:satır kanıtlarıyla belgelendi; hiçbir iddia mantık yürütmesine değil, okunabilir kaynak koduna dayanıyor.

154	~835	223	100 ms
dump.rar Python dosyası	C++ tarafında Python binding'i	CCharacter sunucu fonksiyonu	FUNC_WAIT senkron adımı

Ana bulgular:

B1 — Tek ayak havada yürümenin kök nedeni bulundu. Botlar hareketi `chr.SetPixelPosition` ışınlaması + tek karelik tuş simülasyonuyla yapıyor; hareket vektörü hiç 1.0'ı aşmadığı için istemci durumunu FUNC_MOVE (yürüyor) yerine FUNC_WAIT (duruyor) olarak bildiriyor. Sunucu FUNC_WAIT'i anlık konum kabulü olarak işliyor; izleyici istemciler ise her FUNC_WAIT paketinde yürüyüş animasyonunu başlatıp hedefe anında varınca durduruyor. Yürüyüş döngüsü her 100 ms'de ilk karelerinden yeniden başlıyor — tam bir adım süresi asla dolmuyor.

B2 — Bu davranış bilinçli. LegendySF kaynaklarında ENABLE_TP_SPEED_CHECK yorum satırında olduğu için FUNC_WAIT ile bildirilen konumların mesafe kontrolü yapılmıyor; küçük adımlar dizisi sunucuda ışınlama hızı üretirken tek seferlik büyük sıçrama reddedilir. Animasyon bozukluğu bu yöntemin kabul edilmiş yan etkisi.

B3 — SendCharacterStatePacket Python'a kapalı. İstemci bu paketi kendi event handler'larıyla (OnWaiting/OnMoving/OnMove) ve 100/300 ms gaz kelebeğiyle otomatik gönderiyor; botun paketi elle göndermesi gerekmiyor ve gerekemiyor. Teorinizdeki 'adım atarken daha az döngüde yollama' gözlemi dolaylı olarak doğru: FUNC_WAIT yolu hem daha seyrek hem daha ucuz.

B4 — C++/Python sınırı üç kata ayrıldı. Saf Python ile yapılabilenler (hareket, ışınlama, saldırı döngüsü, eşya/sohbet/lonca işlemleri), özel C++ modül gerektirenler (m2BotLib gibi: ham CG_ATTACK paketi) ve saf Python'da imkânsız olanlar (paket koklama, DLL injection, bellek erişimi).

B5 — Sunucu tarafı tespit yolları tanımladı. Bölüm 8'de FUNC_WAIT frekans/mesafe istatistiği, hareket vektörü-FUNC tutarsızlık denetimi ve ENABLE_TP_SPEED_CHECK etkinleştirme dahil altı somut önlem var.

> **Sorunuzun kısa cevabı:** Botlar SendCharacterState'i adım atarken YOLLAMIYOR değil; YOLLUYOR ama FUNC_WAIT olarak. Adım animasyonunun tamamlanmaması paketin gönderilmemesinden değil, paketin TÜRÜNDEN ve konum bilgisinden kaynaklanıyor: her FUNC_WAIT izleyici istemcide 'bir adım başlat, hedefe anında var, dur' demek. Bu döngü yürüme animasyonunu sürekli ilk karelerde (tek ayak kalkık poz) yeniden başlatır. Detay: Bölüm 7.

2. Kaynaklar ve Yöntem

Analiz üç kaynak katmanının birbirini doğrulaması üzerine kuruldu. Hiçbir katman tek başına yeterli değil: C++ kaynağı mekanizmanın tanımını veriyor, Python dump'ı bu mekanizmanın sahadaki gerçek kullanımını gösteriyor, önceki MyGameArea raporları ise terminolojiyi ve paket typedef sözlüğünü sağlıyor. Aşağıdaki tablo her kaynağın kapsam ve güven düzeyini özetliyor.

Kaynak	İçerik	Boyut	Bu rapordaki rolü
LegendySF-V2 (DrLegendy)	Metin2 serverfiles: Legendy.Source/ClientSource (627 .cpp/.h), Legendy.Source/GameSource (game/db/common/lib*), Metin2Client (derlenmiş istemci)	4.8 GB 73.679 dosya	Mekanizmanın C++ tanımı; paket struct'ları; sunucu işleyicileri
dump.rar	İstemci içinden pydumpall ile toplanan Python katmanı: client çekirdek modülleri, botlar, hackler, UIScript dump'ları + INDEX.txt kanıt zinciri	156 dosya 3.3 MB	Botların gerçek hareket metodları; kanıt kodu (darealfreakbot, eiglib, WalkToMobs)
MyGameArea	Önceki oturumların raporları (DEV_Report.md, DEV_LOGIN_FLOW.md, DEV_GIL_ANALYSIS.md), MaxExecutor, WORKLOG	16 oturum	Terminoloji, TPacketCGMove typedef sözlüğü, Boost fonksiyon analizi ile uyumluluk

Tablo 1: Analiz kaynakları ve rolleri.

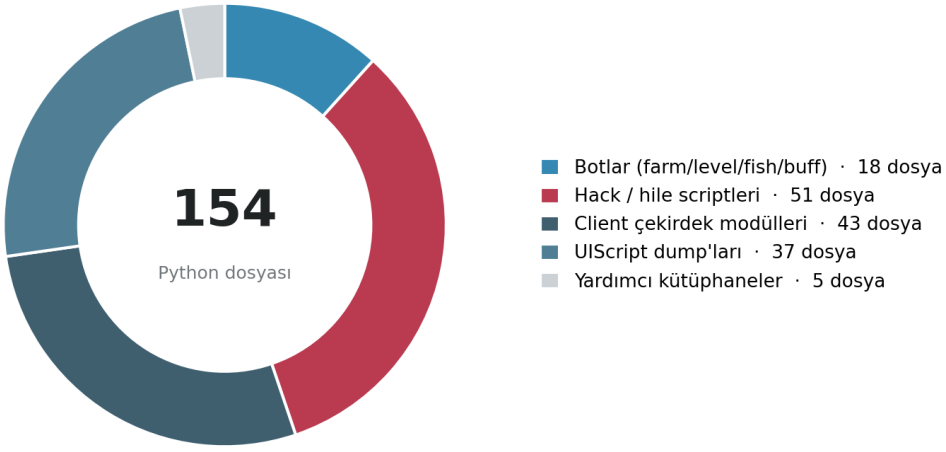
Yöntem dört adımlıydı. İlk adımda dump.rar açıldı, 156 dosya kategorilere ayrıldı ve her dosyanın çağırdığı C modülü API'leri (chr.*, player.*, net.* …) sayaçla çıkarıldı. İkinci adımda LegendySF-V2 client kaynağında hareketle ilgili her fonksiyon zinciri uçtan uca izlendi: Python binding'inden (PythonCharacterModule.cpp) CInstanceBase hareket motoruna (InstanceBaseMovement.cpp), oradan CPythonNetworkStream::SendCharacterStatePacket'e ve animasyon motoruna (GameLib/ActorInstance*.cpp) kadar. Üçüncü adımda sunucu tarafı CInputMain::Move işleyicisi ve GC_MOVE yayını okundu. Dördüncü adımda topluluk kaynakları (metin2.dev protokol tartışmaları, elitepvpers bot yayınları) bulguların genellenebilirliğini test etti; elitepvpers'taki Eylül 2025 tarihli Auto Hunt yayını, dump'taki botlarla birebir aynı hareket desenini kullanıyor.

Kanıt standardı şu: rapordaki her mekanik iddia, parantez içinde dosya adı ve satır numarasıyla doğrulanabilir olmalıdır (örn. PythonNetworkStreamPhaseGame.cpp:1152). LegendySF-V2'nin 'sparse' klonu üzerinde çalışıldığı için satır numaraları, deponun güncel main dalına aittir; kendi kopyanızda fonksiyon adlarıyla arama yapılarak birebir bulunabilir. Satır kanıtlarının tam listesi Bölüm 9'daki

kaynakçadadır.

3. dump.rar Envanteri: Python Katmanı

INDEX.txt, arşivin nasıl toplandığını kanıt zinciriyle anlatıyor: pydumpall v4.1, oyun istemcisinin içinde çalışıp pack.Get yoluyla 73 Python dosyasını diske yazmış, sys.modules içinde 55 modül görmüş ve bunlardan 31'inin 'KAYNAKSIZ(C)' — yani istemci binary'sine gömülü C modülleri — olduğunu tespit etmiş. Bu liste (app, background, chat, chr, chrmgr, dbg, effect, event, fly, grp, grpImage, grpText, guild, ime, item, messenger, net, nonplayer, pack, player, profiler, quest, safebox, sash, skill, snd, systemSetting, textTail, wndMgr) aynı zamanda Python tarafının erişebildiği düşük seviyeli API yüzeyinin sınırlarını da çiziyor. Arşivdeki 154 Python dosyasının kategori dağılımı aşağıda.



Şekil 1: dump.rar — 154 Python dosyasının kategori dağılımı (pydumpall v4.1 çıktısı).

Botlar (18 dosya). Kategori, 284 satırlık basit WalkToMobs.py'den 9511 satırlık darealfreakbot.py'ye kadar uzanıyor. Hareket için üç metod karışık kullanılıyor: chr.MoveToDestPosition (animasyonlu yürüyüş), chr.SetPixelPosition (ışınlama) ve player.SetAttackKeyState (sürekli saldırı döngüsü). darealfreakbot ve pasha37-levelbot, kendileriyle aynı kod tabanını paylaşan m2BotLib/api özel modüllerini import ediyor; bu modüller C++ DLL olarak enjekte ediliyor ve saf Python'da olmayan ham paket fonksiyonlarını (örn. m2BotLib.SendAttackPacket) sağlıyor.

Hack scriptleri (51 dosya). Teleport (teleport1-4, Teleporthack), çoklu hile paketleri (multihack, multihack2-4, kagb-multihack, rajmultihack, vbmultihack), hedef odaklı araçlar (moblock, MetinFinder, ShopScanner) ve Türkçe topluluk üretimi araçlar (efsunbotu, itembasma, padmak, okcuhack, metinbulucu) bu sınıfta. Crap_Levelbot'taki ışınla-ve-topla deseni (±2000 piksel sıçramalar) Bölüm 7'de adım animasyonu sorusunun kanıt dosyalarından biri.

Client çekirdek modülleri (43 dosya) ve UIScript dump'ları (37 dosya). Bunlar istemcinin kendi Python katmanı: game.py, interfacemodule.py, networkModule.py, intro* fazları ve ui* pencere sınıfları. UIScript

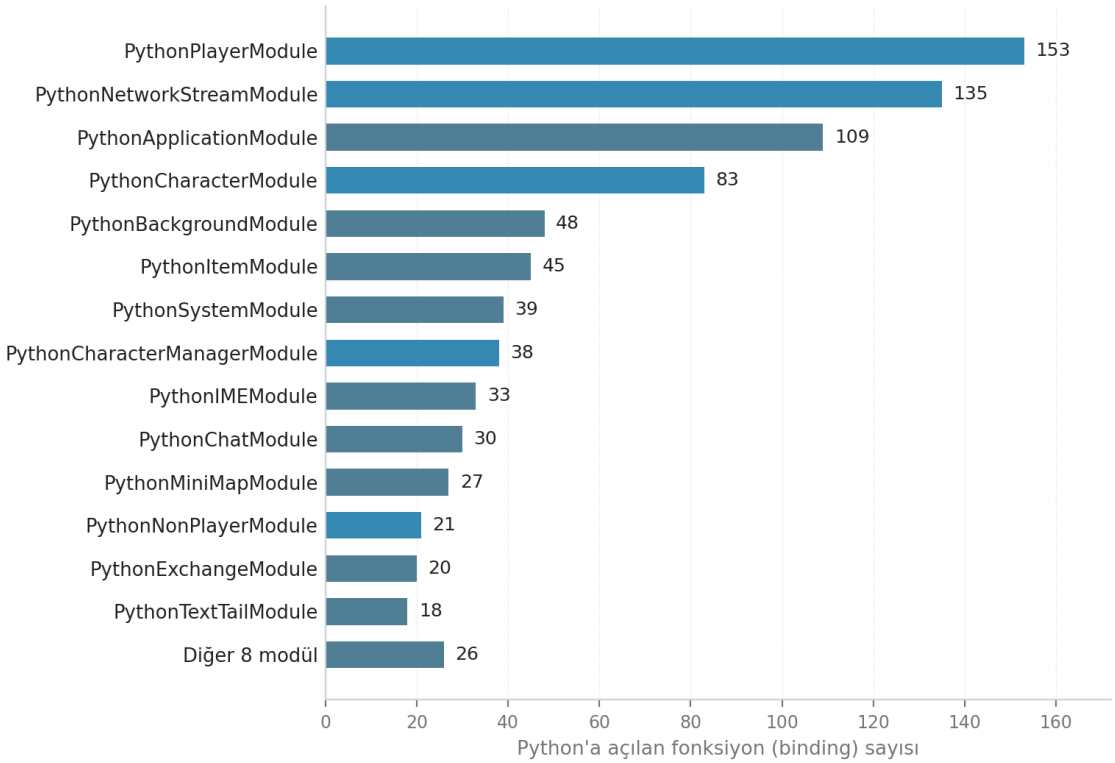
klasöründen dump edilenler (ui__UIScript_*.py.py çift uzantısı pydumpall'ın adlandırma izidir) istemcinin arayüz tanımlarıdır; bu iki kategori bot yazımında 'meşru API' referansı olarak kullanılır. **Yardımcı kütüphaneler (5 dosya)** ise eiglib gibi topluluk fonksiyon koleksiyonlarıdır — eiglib.WalkVID/WalkCOORD/SendAttack üçlüsü, bu raporun hareket sorusunu çözen mini referans bot niteliğindedir.

```
def SendAttack(vid):                        # eiglib.py:144-153
    Distance = player.GetCharacterDistance(vid)
    myx, myy, myz = player.GetMainCharacterPosition()
    mobX, mobY, mobZ = chr.GetPixelPosition(vid)
    if Distance <= 210:
        chr.SetRotation(GetDegree(mobX, mobY, myx, myy))
        player.SetAttackKeyState(TRUE)      # saldırı döngüsü ACIK
    else:
        player.SetAttackKeyState(FALSE)     # saldırı döngüsü KAPALI
        WalkVID(vid)                        # -> chr.MoveToDestPosition
def WalkVID(vid):                           # eiglib.py:110-125
    x, y, z = chr.GetPixelPosition(vid)
    ...                                     # hedefe 135 px kala dur
    chr.MoveToDestPosition(int(myVid), int(aimx), int(aimy))
```

Kod 1: eiglib.py — botların standart yaklaş-saldır döngüsü (animasyonlu yürüyüş deseni).

4. LegendySF-V2 Envanteri: C++ Katmanı

ClientSource klasörü, klasik Metin2 istemci yapısının tamamını içeriyor: UserInterface (oyun mantığı ve Python modül binding'leri), GameLib (aktör/animasyon motoru), EterPythonLib (Python gömme), EterPack, EterLib, SpeedTreeLib ve diğer grafik kütüphaneleri. Python'a açılan API yüzeyi 22 *Module.cpp dosyasında tanımlı ve toplam yaklaşık 835 fonksiyon barındırıyor. Aşağıdaki dağılım, dump'taki botların hangi modüllerle hangi kapıdan konuştığını da açıklıyor: player (153) ve chr (83) modülleri hareket/saldırı için, net (135) paket gönderimi için, item (45) ve chat (30) botların ekonomi/iletişim işlemleri için yoğun kullanılıyor.



Şekil 2: LegendySF-V2 ClientSource — modül başına Python binding sayıları (toplam ~835).

Bu binding'lerin botlar için kritik önemi, dump'taki KAYNAKSIZ(C) listesiyle birebir örtüşmesi. pydumpall'ın göremediği chr, player, net, chrmgr modülleri tam olarak bu C++ dosyalarında yaşıyor: örneğin chr.MoveToDestPosition Python çağrısı PythonCharacterModule.cpp:1179'daki chrMoveToDestPosition fonksiyonuna düşer ve pCharacterInstance->NEW_MoveToDestPixelPositionDirection(...) çağırır (satır 1196). Yani botların kullandığı her Python API'sinin arkasındaki C++ davranışı, bu depoda tam metinle okunabilir durumda.

Sunucu tarafında ise GameSource/game/src altında CHARACTER sınıfı (char.cpp, 223 fonksiyon), ağ giriş katmanı CInputMain (input_main.cpp, 52 paket işleyicisi; HEADER_CG_MOVE dahil) ve hareket/animasyon ile ilgili yardımcı birimler (char_state.cpp 27 fonksiyon) yer alıyor. Sunucu, TPacketCGMove paketini CInputMain::Move ile işler (input_main.cpp:1530) ve sonucu TPacketGCMove olarak PacketAround ile çevredeki herkese yayar (satır 1687). Bu iki taraf arasındaki sözleşme Bölüm 6'da katman katman çözülüyor.

Katman	Sınıf / Dosya	Fonksiyon sayısı	Rapor için önemi
Client	CPythonPlayer (PythonPlayerModule.cpp)	153 binding	SetAttackKeyState, hareket girdi durumu; botların ana API'si
Client	CPythonNetworkStream (PythonNetworkStreamModule.cpp)	135 binding	Paket gönderim yüzeyi; SendCharacterStatePacket BURADA DEĞİL
Client	CInstanceBase (InstanceBase*.cpp)	3523 satır	StateProcess: FUNC_WAIT/FUNC_MOVE izleyici davranışı (satır 1432-1490)
Client	CActorInstance (GameLib/ActorInstance*.cpp)	909 satır (Motion)	SetLoopMotion, __ClearMotion, WALK/RUN animasyon döngüsü
Server	CHARACTER (char.cpp)	223	Goto/Move/Stop sunucu hareketi; GetLimitPoint kontrolü
Server	CInputMain (input_main.cpp)	52 işleyici	CG_MOVE validasyonu: hız hack zaman denetimi, TP mesafe kontrolü

Tablo 2: LegendySF-V2'nin hareket analizi için kritik C++ birimleri.

5. C++ vs Python Karşılaştırma Matrisi

Sorunuzun ikinci ana ekseni: 'C++'da yapılabilenlerin hangilerini Python'da yapabiliriz?' Bu sorunun kesin cevabı üç kategoriye ayrılıyor. Kategori A: Python'un gömülü modülleriyle DOĞRUDAN yapılabilenler — istemci içi Python'da çalışan her bot için bedava. Kategori B: saf Python ile yapılıp görünmeyen ama özel C++ modülü (m2BotLib/api gibi enjekte edilmiş DLL) gerektirenler. Kategori C: istemci içi Python'da prensip olarak imkânsız olanlar. Tablo 3-5 bu üç kategorinin tam envanteridir; 'kanıt' sütunundaki dosya:satır referansları LegendySF-V2 kaynağına işaret eder.

5.1 Kategori A — Saf Python ile DOĞRUDAN yapılabilenler

Yetenek	Python API (modül.fonksiyon)	C++ karşılığı ve kanıtı	Not
Animasyonlu yürüyüş	chr.MoveToDestPosition(vid,x,y)	chrMoveToDestPosition → NEW_MoveToDestPixelPositionDirection (PythonCharacterModule.cpp:1179)	NEW_Goto tetikler; WALK/RUN animasyonu çalar
Anlık ışınlama (client)	chr.SetPixelPosition(x,y,z)	SCRIPT_SetPixelPosition / NEW_SetPixelPosition (InstanceBaseTransform.cpp:8-17)	Paket GÖNDERMEZ; sadece yerel konum
Sürekli saldırı döngüsü	player.SetAttackKeyState(True/False)	CPythonPlayer::SetAttackKeyState → m_isAtkKey (PythonPlayerInputKeyboard.cpp:6-20)	Döngü Update'te m_isAtkKey'i işler
Yön/dönüş	chr.SetRotation / chr.SetDirection	InstanceBaseTransform.cpp:24-65	bRot için 5° çözünürlük
Konum okuma	chr.GetPixelPosition(vid), player.GetMainCharacterPosition()	PythonCharacterModule.cpp; PythonPlayerModule.cpp	Mesafe/yol hesaplarının kaynağı
Hedef seçimi	chr.SelectInstance(vid), player.SetTarget(vid)	PythonCharacterModule.cpp:1112 civarı	Saldırı öncesi hedef kilidi
Görüş alanı taraması	chr.GetInstanceType, chr.GetRace, nonplayer.GetLevelByVID	PythonCharacterManagerModule / PythonNonPlayerModule (21 binding)	WalkToMobs.py VID tarama yöntemi
Eşya işlemleri	net.SendItemUsePacket, net.SendItemMovePacket, net.SendItemPickUpPacket	PythonNetworkStreamModule.cpp:1984-1997	Pickup/bot envanter döngüleri
Sohbet/fısıltı	net.SendChatPacket, net.SendWhisperPacket	PythonNetworkStreamModule.cpp:2003-2006	Spambotların çekirdeği
Depo/pazar/takas	net.SendSafebox*, net.SendShop*, net.SendExchange*	PythonNetworkStreamModule.cpp:2010-2048	ShopScanner/shopcreator'in API'si
Hız değeri değişimi (görünüm)	chr.SetMoveSpeed(vid,val)	chrSetMoveSpeed (PythonCharacterModule.cpp:1389) → CActorInstance::SetMoveSpeed	Sunucu MOV_SPEED'ini değiştirmez; yerel animasyon hızı
UI/zoom/kamera	wndMgr.*, app.SetCamera*, grp.*	PythonApplicationModule (109 binding)	zoomhack'in meşru yolu

Tablo 3: Kategori A — gömülü modüllerle doğrudan erişilebilen C++ yetenekleri.

5.2 Kategori B — Özel C++ modülü (enjekte DLL) gerektirenler

Yetenek	Özel modül karşılığı	Neden saf Python yetmiyor	Kanıt
Ham CG_ATTACK saldırı paketi	m2BotLib.SendAttackPacket(vid)	net modülünde SendAttackPacket binding'i YOK (method tablosunda geçmiyor)	dump: darealfreakbot.py:665; net modülü envanteri
SendCharacterStatePacket'i elle çağırma	api/m2BotLib sarmalayıcıları veya Boost tarzı kompozit	Fonksiyon CPythonNetworkStream'de VAR (PythonNetworkStreamPhaseGame.cpp:1152) ama Python'a AÇILMAMIŞ	DEV_Report.md Boost analizi ile uyumlu
Ham paket gönderim/şablon	Özel modüller (api.SetHandlers vb.)	net.Send* ailesi hazır paketler sunar; serbest byte akışı için binding yok	PythonNetworkStreamModule.cpp method tablosu
Ağ/kanal yönetimi	ServerInfo + özel modüller	Sunucuya ikinci bağlantı açmak için gerekli	eiglib.ConnectToChannel (eiglib.py:81)

Tablo 4: Kategori B — özel C++ DLL gerektiren yetenekler.

5.3 Kategori C — İstemci içi saf Python'da İMKÂNSIZ olanlar

Yetenek	Neden imkânsız	Gerçek uygulama yolu
Paket koklama (GC/CG trafik izleme)	Python katmanı paket akışını görmez; CNetworkStream C++ katmanında şifreler	C++ enjeksiyonu (MaxExecutor yaklaşımı) veya sürücü/seviye sniffer
DLL injection / fonksiyon hook	Python kendi sürecini hook edemez	C++ DLL (MaxExecutor.dll yaklaşımı: WndProc + ana iş parçacığı)
Bellek okuma/yazma (ReadProcessMemory)	Python modülleri ptr erişimi sunmaz	Harici araç veya özel C modülü
Sunucu taraflı istismar	Tüm sunucu doğrulaması C++ tarafında	Kaynak kodu olan sunucuda sistem yazımı (LegendySF sahibiyseniz)

Tablo 5: Kategori C — saf Python sınırı ve gerçek uygulama yolları.

Matrisin pratik sonucu şudur: bot davranışlarının büyük çoğunluğu Kategori A ile üretilebilir ve dump'taki botlar da aynen böyle yapar. Kategori B'nin iki kalemi, saldırı paketinin ham halidir ve dump topluluğu bunu m2BotLib gibi enjekte edilmiş C++ modülleriyle çözmüştür; MaxExecutor'ınızın pylib yaklaşımı, aynı ihtiyacı hook'suz ana iş parçacığı yürütmesiyle karşılıyor. Kategori C, istemci içi Python'un doğal sınırıdır ve bu sınıra girmek isteyen her projenin C++ tarafına geçmesi zorunludur.

6. Hareket Protokolü: Katman Katman

Adım animasyonu sorusunu çözmek için önce hareketin sözleşmesini doğru kurmak gerek. Metin2'de hareket tek bir paket çiftiyle yürür: istemciden sunucuya CG_MOVE (HEADER_CG_MOVE = 7, struct TPacketCGMove), sunucudan çevreye GC_MOVE (struct TPacketGCMove). Paketin ismi source'ta 'character move' olsa da, SendCharacterStatePacket fonksiyonunun içinde build edildiği için hile topluluğu buna 'character state' der; ikisi aynı şeydir ve bu raporda 'state paketi' olarak geçer.

Alan	Tip	İçerik	Kanıt
bHeader	BYTE	HEADER_CG_CHARACTER_MOVE (=7)	PythonNetworkStreamPhaseGame.cpp:1164
bFunc	BYTE	Hareket fonksiyonu: FUNC_WAIT/FUNC_MOVE/FUNC_ATTACK/FUNC_COMBO/FUNC_MOB_SKILL/FUNC_EMOTION (0-5) veya FUNC_SKILL|motion (0x80+)	InstanceBase.h:70-76; input_main.cpp:1537-1552
bArg	BYTE	Func'a göre argüman: combo vuruş indeksi, skill arg vb.	InstanceBase.cpp:1503,1523
bRot	BYTE	Dönüş açısı / 5 (5 derece çözünürlük; istemci fDstRot'u normalize eder)	PythonNetworkStreamPhaseGame.cpp:1158-1167
lX, lY	LONG	Hedef piksel konumu; __LocalPositionToGlobalPosition ile dünya koordinatına çevrilir	satır 1168-1174
dwTime	DWORD	ELTimer_GetServerMSec() — sunucu saat tabanlı zaman damgası	satır 1170
dwDuration	DWORD	SADECE GC_MOVE'da: hareket süresi; FUNC_MOVE'da GetCurrentMoveDuration(), diğerlerinde 0	input_main.cpp:1685

Tablo 6: TPacketCGMove / TPacketGCMove alan haritası (CG ve GC versiyonları aynı çekirdeği paylaşır, GC tarafında dwVID eklenir).

FUNC sabitleri ve anlamları. Client kaynağındaki enum hareket fonksiyonlarının tamamını tanımlar: FUNC_WAIT=0 (dur), FUNC_MOVE=1 (yürü), FUNC_ATTACK=2 (normal saldırı), FUNC_COMBO=3 (kombinasyon saldırısı), FUNC_MOB_SKILL=4, FUNC_EMOTION=5 ve FUNC_SKILL=0x80|motion (beceri). Sunucu bFunc >= FUNC_MAX_NUM && !(bFunc & 0x80) durumunda paketi reddeder (input_main.cpp:1537) — yani 0-5 arası değerler ve 0x80 maskeli beceri değerleri geçerlidir.

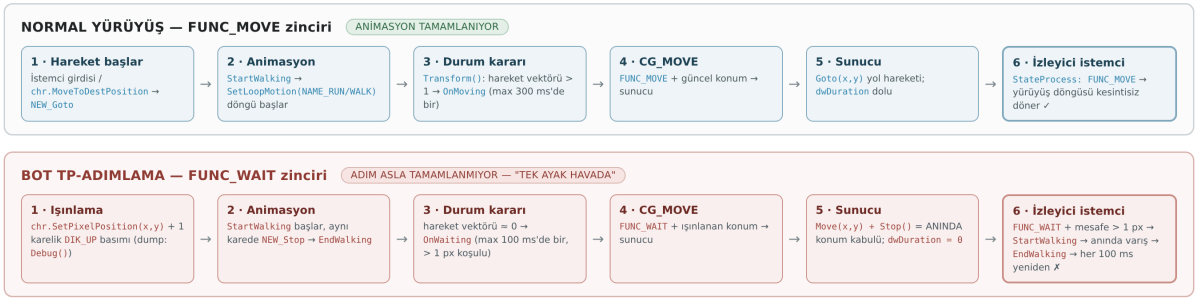
6.1 İstemci tarafı: kim, ne zaman, hangi FUNC'ı gönderiyor?

Event handler	Gönderilen paket	Gaz kelebeği (throttle)	Kanıt (PythonPlayerEvent Handler.cpp)
OnMove (hareket başlangıcı)	FUNC_MOVE + güncel konum	Anında; her iki zamanlayıcıyı da ileri sarar	satır 76-86
OnMoving (hareket sürerken)	FUNC_MOVE + güncel konum	En fazla 300 ms'de bir	satır 62-74
OnWaiting (beklerken)	FUNC_WAIT + güncel konum	En fazla 100 ms'de bir VE son bildirimden beri > 1 px yer değişimi varsa	satır 38-60
OnStop / OnWarp	FUNC_WAIT	Anında	satır 88-100
OnAttack	FUNC_COMBO + vuruş indeksi	Her vuruşta	satır 102-118
OnUseSkill	FUNC_SKILL|motion + arg	Her beceri kullanımında	satır 120-124

Tablo 7: İstemci durum makinesinin paket programı — 100/300 ms gaz kelebekleri sorunuzun merkezindedir.

Bu tablonun en kritik satırı OnWaiting'in koşuludur: 'son bildirimden beri 1 pikselden fazla yer değişimi varsa'. Peki 'yer değişimi' neye göre ölçülüyor? Cevap CInstanceBase::Transform'ta (InstanceBase.cpp:1994-2015): her karede, IsWalking() doğruysa, animasyon motorunun o karede ürettiği hareket vektörünün uzunluğu (m_GraphicThingInstance.GetMovementVectorRef) 1.0'dan büyükse OnMoving, değilse OnWaiting çağrılır. Yani istemci, 'yürüyor muyum?' kararını paket türüne değil, animasyon motorunun ürettiği GERÇEK yer değiştirmeye göre verir. Işınlama yapan karakterde bu vektör sıfıra yakındır — istemci ışınlanan karakteri 'duruyor' sayar. Bu tek cümle, Bölüm 7'deki mekanizmanın tohumudur.

6.2 Sunucu tarafı: FUNC_MOVE ile FUNC_WAIT arasındaki dev fark. CInputMain::Move (input_main.cpp:1530-1700) iki fonksiyon değerini tamamen farklı işler: FUNC_MOVE için ch->Goto(lX, lY) çağırır — sunucu karakteri yol boyunca hareket ettirir, hız limitine (GetLimitPoint) bağlıdır ve yayına dwDuration = GetCurrentMoveDuration() ile çıkar. FUNC_WAIT (ve saldırı türleri) içinse ch->Move(lX, lY) + ch->Stop() çağırır — karakter ANINDA bildirilen konuma taşınır ve dwDuration = 0 ile yayılır. Yani FUNC_WAIT, protokolün 'buradayım' ışınlama mekanizmasıdır; sunucu istisnasız kabul eder (tek kontrol: 7 saniye sonrası için dwTime tabanlı hız hack zaman denetimi, satır 1591-1612).



Kaynak: LegendySF-V2 ClientSource (PythonPlayerEventHandler.cpp:38-94 · InstanceBase.cpp:1432-1490 · InstanceBaseMovement.cpp:250-290 · ActorInstance.cpp:209-225) ve GameSource (input_main.cpp:1530-1687). İzleyici tarafta her FUNC_WAIT paketi yürüyüş döngüsünü ilk karelerinden başlatır; WALK/NAME_RUN döngüsü bir kez bile tam bir adım süresini tamamlayamaz — karakter ayakları havada/kayarak yürüyor görünür.

Şekil 3: Normal yürüyüş (FUNC_MOVE) ve bot TP-adımlama (FUNC_WAIT) zincirlerinin uçtan uca karşılaştırması.

7. Adım Atma Animasyonu Mekanizması (Çekirdek Bölüm)

Bu bölüm, sorunuzun — 'bazı botlar adım atma fonksiyonuna bir şey yapıyor; karakterin tek ayağı havaya kalkıyor, adım atma animasyonu tamamlanmadan öyle yürüyor; buna neden olan şey nedir?' — tam cevabıdır. Mekanizmayı dört katmanla çözeceğiz: animasyon motoru, durum makinesi, bot deseni ve izleyici tarafı. Her katmanda tek bir dosya:satır kanıtı yeterli olacaktır çünkü zincirin tüm halkaları zaten Bölüm 6'da kuruldu.

7.1 Animasyon motoru: döngü animasyonu nasıl başlar, nasıl sıfırlanır? CActorInstance::Move (GameLib/ActorInstance.cpp:209-219), m_isWalking bayrağına bakar: yürüme modunda SetLoopMotion(CRaceMotionData::NAME_WALK, 0.15f, m_fMovSpd), koşma modunda NAME_RUN çağrılır. SetLoopMotion (ActorInstanceMotion.cpp:290-320) her çağrıda önce __ClearMotion() çalıştırır (satır 307) — bu fonksiyon (satır 627-637) mevcut kareyi dwcurFrame=0'a sıfırlar ve animasyon kuyruğunu temizler. Başka bir deyişle: SetLoopMotion her çağrıldığında yürüyüş döngüsü BAŞTAN başlar; 0.15 saniyelik blend süresi, karakteri bekleyiş pozuyla yürüyüş pozunun karışımı bir geçiş karesine sokar — yürüme döngüsünün ilk kareleri, tam olarak ayağın havaya kalktığı adım başlangıcı pozudur.

```
void CActorInstance::Move()                       // ActorInstance.cpp:209
{
    if (m_isWalking)
        SetLoopMotion(CRaceMotionData::NAME_WALK, 0.15f, m_fMovSpd);
    else
        SetLoopMotion(CRaceMotionData::NAME_RUN,  0.15f, m_fMovSpd);
}
void CActorInstance::Stop(float fBlendingTime)    // ActorInstance.cpp:221
{
    __ClearMotion();                              // kare = 0, kuyruk temiz
    SetLoopMotion(CRaceMotionData::NAME_WAIT, fBlendingTime);
}
void CActorInstance::SetMoveSpeed(float fMovSpd)  // ActorInstance.cpp:190
{
    if (m_fMovSpd == fMovSpd) return;
    m_fMovSpd = fMovSpd;
    if (__IsMoveMotion()) { Stop(); Move(); }     // HIZ DEGISIRSE ANIMASYON RESTART
}
```

Kod 2: Animasyon motorunun üç kritik davranışı — başlatma, sıfırlama ve hız değişiminde restart.

7.2 İzleyici tarafı: FUNC_WAIT geldiğinde ne olur? StateProcess (InstanceBase.cpp:1381-1490) ağdan gelen komut kuyruğunu işler. FUNC_WAIT dalı (satır 1434-1466) iki durumu ayırır: yeni konumla mevcut render konumu arasındaki mesafe (fDirLen) 1 pikselden büyükse — karakter YÜRÜYÜŞE başlar (NEW_SetSrcPixelPosition/NEW_SetDstPixelPosition, m_isGoing=TRUE, IsWalking() değilse StartWalking()) ve m_kMovAfterFunc = FUNC_WAIT ayarlanır; yani varınca tekrar DURACAKTIR. Mesafe 1 pikselden küçükse karakter EndWalking() ile durur ve konumu anında sabitler. FUNC_MOVE dalı (satır 1469-1490) ise StartWalking sonrası varışta durmaz — m_kMovAfterFunc FUNC_MOVE kalır, yürüyüş kesintisiz sürer (satır 1791-1794 boş case).

Şimdi iki zinciri birleştirin. Botun karakteri sunucuya FUNC_WAIT'le adım adım konum bildiriyor (aşağıda 7.3'te nasıl olduğuna dair kanıt var). İzleyici istemci her FUNC_WAIT'i şöyle işliyor: yeni konum > 1 px uzakta olduğu için yürüyüş animasyonu BAŞLATILIR — SetLoopMotion ile WALK/RUN döngüsü kare 0'dan, yani ayak havaya kalkan ilk adım pozundan başlar. Bot 100 ms sonra zaten o konuma ışınlanmış olduğu için hareket süresi fiilen anlıktır (dwDuration=0, kısa mesafe): karakter hedefe ilk birkaç karede varır ve EndWalking() ile durur. Hemen ardından yeni FUNC_WAIT gelir ve aynı şey tekrarlanır. Sonuç: izleyicinin ekranında karakter yürüyor GİBİ görünür ama yürüyüş döngüsü hiçbir zaman bir tam adım süresini (normalde yaklaşık 0.5-1.0 s) tamamlayamaz — karakter sürekli adımın İLK karelerinde donar: tek ayak havada, ayaklar yer tutmadan kayarak ilerler.

> **Mekanizma tek cümlede:** FUNC_WAIT adımları izleyici istemcide 'adım başlat → anında var → dur' mikro-döngüsü üretir; bu döngü yürüyüş animasyonunu her 100 ms'de kare 0'dan yeniden başlattığı için tam adım karesi asla oynatılamaz ve karakter tek ayağı havada, kayarak yürüyen bir görünüme bürünür.

7.3 Bot deseni: dump kanıtı. Bu davranışı dump'taki botların kodunda doğrudan görmek mümkündür. darealfreakbot.py'de hareket fonksiyonu bir mod anahtarıyla çalışır: IsWalkToMob() doğruysa chr.MoveToDestPosition (meşru yürüyüş), değilse chr.SetPixelPosition ile ışınlanır VE ardından Debug()

çağrılır. Debug() iki satırdan oluşur: player.SetSingleDIKKeyState(app.DIK_UP, TRUE) ve hemen ardından (..., FALSE) — yani yukarı ok tuşuna TEK KARE basma simülasyonu. Bu tek karelik basım NEW_SetMultiDirKeyState yoluyla NEW_MoveToDirection'a gider: StartWalking() çağrılır (yürüyüş animasyonu başlar), 300 piksellik kısa bir hedef konur; tuş bırakımı aynı tıkta NEW_Stop() → EndWalking() üretir (PythonPlayerInputKeyboard.cpp:22-89; InstanceBaseMovement.cpp:30-45,292-335). Botun kendi ekranında karakter bir yarım adım oynatır; ağ tarafında ise Transform() hareket vektörünü ölçer ve ölçtüğü şey 'hareket yok' olduğundan FUNC_WAIT yayınlanır. Elitepvpers'taki Eylül 2025 tarihli Auto Hunt yayınında birebir aynı desen (SetPixelPosition + self.Debug() + TeleportState=1) görülür — teknik, topluluk tarafından en az on yıldır bilinen standarttır.

```
if self.IsWalkToMob():                          # darealfreakbot.py:688-693
    chr.MoveToDestPosition(int(myVid), int(self.aimx), int(self.aimy))
else:
    chr.SelectInstance(myVid)
    chr.SetPixelPosition(int(x), int(y), int(z))   # IŞINLAMA MODU
    self.Debug()
def Debug(self):                                # darealfreakbot.py:695-697
    player.SetSingleDIKKeyState(app.DIK_UP, TRUE)   # yukari ok 1 kare BAS
    player.SetSingleDIKKeyState(app.DIK_UP, FALSE)  # -> yarim adim + state sync
```

Kod 3: darealfreakbot.py — ışınla + tek karelik tuş deseni (dump kanıtı; elitepvpers Auto Hunt 2025 ile aynı).

7.4 Zaman çizelgesi: izleyicideki 100 ms döngüsü. Aşağıdaki tablo, TP-adımlama yapan bir botu izleyen bir oyuncunun istemcisinde geçen süreci kare kare koyar. Tablodaki T noktaları, OnWaiting'in 100 ms gaz kelebeğiyle (PythonPlayerEventHandler.cpp:44) botun yeni bir FUNC_WAIT paketini sunucuya verdiği anlardır; sunucu bunu anında kabul edip PacketAround ile yaydığı için izleyiciye ulaşım gecikmesi yalnızca ağ gecikmesidir.

Zaman	Bot istemcisi	Sunucu	İzleyici istemci
T+0	SetPixelPosition ile 20 px ışınlanır; Debug() yarım adım oynatır	FUNC_WAIT alır: Move(x,y)+Stop() — karakter anında yeni konumda	GC_MOVE FUNC_WAIT alır: 20 px uzak hedef → StartWalking (kare 0, ayak kalkar)
T+~20 ms	Hareket vektörü ölçülür: ≈ 0 → OnWaiting	dwDuration=0 ile yaydı; karakteri Goto simülasyonuna sokmadı	Karakter hedefe ilk karelerde varır (kısa mesafe) → EndWalking (dur, bekleyiş pozu)
T+100 ms	OnWaiting: > 1 px değişim VAR → yeni FUNC_WAIT	Yine Move+Stop, anında kabul	Yeniden StartWalking → kare 0'dan yarım adım → dur
T+200 ms ...	Döngü sürer (yeni ışınlamalar)	Her adım ayrı ayrı meşru görünür	Yürüyüş döngüsü hiçbir tam adım süresini tamamlayamaz: TEK AYAK HAVADA, kayarak yürüme

Tablo 8: TP-adımlama botunu izleyen oyuncu tarafında adım animasyonunun asla tamamlanmama döngüsü.

7.5 Teorinizin değerlendirmesi. 'SendCharacterState'i adım atarken gönderdetmemek için mi yapıyorlar; botlar SendCharacterState'i canavarlara sürekli yollatır, adım atarken daha az döngüde yollarsa...' — teorinin iki parçası var ve ikisi de kısmen doğru, ama gerçek mekanizma daha incelikli. Doğru olan: botlar saldırı sırasında durum akışını yoğun tutar (her vuruşta FUNC_COMBO + CG_ATTACK + itme sonrası

SyncPosition), adım/ışınlanma sırasında ise akış seyrekleşir — FUNC_WAIT sadece 100 ms'de bir ve yalnızca 1 pikselden fazla kayma varsa gider. Yanlış olan: paketin TÜMÜNÜ bastırmak gerekmez; bastırsaydınız sunucu konumunuzu hiç bilemez, karakteriniz sunucuda eski yerde sayardı ve saldırılar reddedilirdi. Gerçek numara, paketi FUNC_MOVE yerine FUNC_WAIT olarak göndermektir — SetPixelPosition yaptığınız anda Transform()'ın hareket vektörü ölçümü bunu sizin yerinize otomatik yapar. Yani 'adım atarken durum paketini az yollamak' yan etki; 'yürüme durumunu hiç bildirmemek (FUNC_MOVE akışını hiç açmamak)' kasıtlı seçim; 'tek ayak havada' görünümü ise bu seçimin izleyici tarafındaki fiyatıdır.

8. Bilinçli mi? Motivasyon ve Sunucu Tespiti

Bu yöntemin bilinçli seçildiği beş bağımsız gerekçeyle desteklenir. Birincisi hız: FUNC_WAIT ışınlaması yol bulma, ivmelenme ve yürüyüş süresi beklemez; bot hedefe anında konumlanır. İkincisi sunucu kabulü: LegendySF-V2 kaynaklarında ENABLE_TP_SPEED_CHECK tanımlaması yorum satırında bulunduğu için (input_main.cpp:1557) FUNC_WAIT konumlarının mesafe denetimi yapılmaz — büyük mesafeler bile kabul edilir; küçük adımlar bu durumda hiç risk üretmez. Üçüncüsü, aynı dosyadaki hız hack zaman denetimi (satır 1591-1612) yalnızca dwTime damlasının sunucu saatine göre geleceğe mi kaçtığına bakar; ışınla-adımlama bu denetime takılmaz. Dördüncüsü algı: tek seferlik büyük ışınlamalar 'blink' diye gözle görülürken, 100 ms'lik küçük adımlar uzaktan sürekli bir kayma gibi algılanır ve kameranın uzağında fark edilmemesi kolaydır — animasyon bozukluğu bunun bilinen ve kabul edilmiş bedelidir. Beşincisi maliyet: FUNC_WAIT paketi kısa, seyrek ve sunucuda yolsuz; FUNC_MOVE akışı ise Goto simülasyonu ve dwDuration hesabı demektir.

Dev prangası notu: DEV_Report.md'de belgelendiğiniz gibi C4US'un Boost bileşimi de aynı sözleşmeyi kullanır — SCRIPT_SetPixelPosition ile yerel taşı, SendCharacterStatePacket ile küçük adımlarla sunucuya bildir; 'sunucu küçük adımları kabul eder, tek seferlik büyük sıçramayı reddeder'. LegendySF'de büyük sıçramayı reddedecek kontrolün devre dışı olduğu görülüyor; yani mevcut haliyle sunucu bu desene karşı tamamen savunmasız. Aşağıdaki tablo, bu savunmasızlığı kapatmak için kaynak kodunuza uygulanabilir altı önlemin teknolojik özetidir.

#	Önlem	Uygulama yeri (LegendySF kaynak)	Yakaladığı desen
1	ENABLE_TP_SPEED_CHECK etkinleştir	input_main.cpp:1557 (#define yorumu) — dist > 25 m (binede 60 m) reddi	FUNC_WAIT ile uzak konum bildirimi; Crap_Levelbot tarzı ±2000 px sıçramalar
2	FUNC_WAIT mesafe bütçesi	CInputMain::Move: son kabul konumuna göre pencere başına toplam yer değişimi limiti	Sık küçük adımlarla birikmiş ışınlama hızı
3	FUNC_WAIT frekans denetimi	CHARACTER::Move/ResetStopTime yanına hedef-sayaç: FUNC_WAIT > N/adet sn şüphe kaydı	OnWaiting 100 ms deseni spam olarak görünür; insan yürüyüşü FUNC_MOVE ağırlıklıdır
4	Hareket vektörü-FUNC tutarlılığı	İstemci side yerine sunucu: FUNC_WAIT bildirimlerinin konum farkı istatistiği; %95'i 1-30 px olan seri = bot imzası	Işınla-adımlama profili (insanlarda FUNC_WAIT uzun duruşlardan sonra gelir)
5	dwTime sıralı ilerleme denetimi	Mevcut SPEEDHACK bloğuna (input_main.cpp:1591-1612) geriye doğru zaman kontrolü ekle	Kayıt-tekrar (replay) ve zaman istismarı
6	GC_MOVE dwDuration tüketim denetimi	İzleyici simülasyonu yerine sunucuda: FUNC_WAIT bildirim aralığı, kabul edilen konumun Goto süresinden kısa mı?	Sunucu hareket simülasyonuyla çelişen anlık konum sıçramaları

Tablo 9: LegendySF sunucusuna önerilen tespit/önlem seti (kaynak koduna spesifik).

9. Uygulama Notları, Sonuç ve Kaynakça

Pratik çıkarımlar üç kolda toplanır. Koldan biri bot tasarımı, biri sunucu savunması, biri de C++'tan Python'a taşıma yol haritası içindir.

Doğal görünümlü hareket isteniyorsa: tek seçenek FUNC_MOVE akışını açık tutmaktır — yani chr.MoveToDestPosition zincirleri (eiglib.WalkVID deseni) kullanmak ve istemcinin OnMoving 300 ms gaz kelebeğine güvenmek. Worklog'da denediğiniz otp (240 px adım + MoveToDestPosition + 1300 ms) tam olarak bu yoldadır ve izleyicide animasyonun düzgün görünmesinin tek yoludur. FUNC_WAIT tabanlı yöntemlerde animasyon bozukluğu kaçınılmazdır; bunun tek istisnası, FUNC_WAIT'in yalnızca DURUŞ anlarında (> 1 px kayma sonrası tek seferlik) kullanılmasıdır — ki bu zaten insan davranışının profiliyle örtüşür. TP-sonrası 'yürüyüşün geri atması' (otp denemenizdeki OptimizeTP sorunu) da aynı kökten beslenir: sunucu FUNC_MOVE'un Goto simülasyonunu tamamlamadan yeni konum kabulü gelirse, simülasyonun kalan mesafesini geri sarar; çözüm, TP sonrası bir FUNC_WAIT konum kabulünün (Move+Stop) ya da bir tam Goto süresinin beklenmesidir.

C++'tan Python'a yol haritası: Kategori A envanteri (Tablo 3) ile başlayın — bot davranışlarının %90'ı orada. Kategori B için iki seçenek var: m2BotLib tarzı ayrı bir enjekte DLL (dump topluluğunun yolu) veya mevcut MaxExecutor mimarinize pylib'e SendAttackPacket benzeri ham paket yardımcıları eklemek — ikincisi hook'suz olduğu için sizin mimarinizle uyumludur. SendCharacterStatePacket'i elle çağırmaya gerek yoktur ve önerilmez: istemci zaten doğru FUNC'ı doğru zamanla gönderir; elle çağrı, throttle'ları atlayıp sunucu tarafında anormal frekans profili yaratır. Kategori C kalemleri (sniffing, injection, bellek) için C++ tarafına geçmeden Python'da zaman harcamayın.

Sonuç. 'Tek ayak havada yürüme' bir bug değil, Metin2 hareket protokolünün FUNC_WAIT anlamlarının ve istemci animasyon durum makinesinin beklenen birleşik sonucudur. Bunu üreten botlar SendCharacterState'i gizlemiyor; onu sunucunun ışınlama olarak yorumladığı biçimde (FUNC_WAIT) kullanıyor ve animasyon motorunun SetLoopMotion/__ClearMotion karesi sıfırlama davranışının üstünden geçiyor. LegendySF sunucusu bu desene karşı altı ölçülebilir önlemle (Tablo 9) kapatılabilir. C++/Python sınırı ise kesin çizgileriyle belirlendi: hareket, saldırı döngüsü ve oyun içi işlemler saf Python'da; ham paket üretimi özel modülde; sistem seviyesi işler C++'da.

9.1 Kod kanıt kaydı (dosya:satır)

Kanıt	Konum	İçerik
SendCharacterStatePacket gövdesi	ClientSource/Client/UserInterface/PythonNetworkStreamPhaseGame.cpp:1152-1189	TPacketCGMove build + bRot/5 + ELTimer_GetServerMSec + global konum dönüşümü
OnWaiting/OnMoving throttles	UserInterface/PythonPlayerEventHandler.cpp:38-94	FUNC_WAIT @100 ms + >1 px koşulu; FUNC_MOVE @300 ms
Transform karar noktası	UserInterface/InstanceBase.cpp:1994-2015	Hareket vektörü > 1 → OnMoving, değilse OnWaiting
StateProcess FUNC_WAIT/FUNC_MOVE	UserInterface/InstanceBase.cpp:1434-1490	İzleyici: >1 px ise StartWalking→varış→EndWalking; FUNC_MOVE kesintisiz
PushTCPState / ağ kuyruğu	UserInterface/InstanceBase.cpp:1289-1330	GC_MOVE komutunun aktöre işlenmesi (m_dwChkTime + network gap)
chr.MoveToDestPosition binding	UserInterface/PythonCharacterModule.cpp:1179-1199	chr → NEW_MoveToDestPixelPositionDirection
chr.SetPixelPosition binding	UserInterface/PythonCharacterModule.cpp:679-702	SCRIPT_/NEW_SetPixelPosition; paket yok
CActorInstance::Move/Stop/SetMoveSpeed	ClientSource/Client/GameLib/ActorInstance.cpp:190-225	NAME_WALK/NAME_RUN/WATCH; hız değişiminde restart
SetLoopMotion + __ClearMotion	ClientSource/Client/GameLib/ActorInstanceMotion.cpp:290-320, 627-637	Kare 0'a sıfırlama — adım başlangıcı pozu
Sunucu CG_MOVE işleyicisi	GameSource/Server/game/src/input_main.cpp:1530-1700	FUNC_MOVE→Goto; FUNC_WAIT→Move+Stop; dwDuration; ENABLE_TP_SPEED_CHECK yorumu; SPEEDHACK zaman denetimi
dump bot deseni	dump/darealfreakbot.py:688-697; dump/eiglib.py:110-153; dump/Crap_Levelbot.py:791-874	SetPixelPosition + DIK_UP; MoveToDestPosition yaklaş-saldır; ±2000 px ışınlama

Tablo 10: Rapor boyunca kullanılan dosya:satır kanıtları (LegendySF-V2 @ main; dump @ pydumpall v4.1).

9.2 Web ve topluluk kaynakları

Kaynak	URL	Katkısı
metin2.dev — Stateful Server and Sync Position between Clients (2022)	https://metin2.dev/topic/28895	CG_CHARACTER_MOVE FUNC_WAIT final konum bildirimi + sync pozisyon protokolünün topluluk doğrulaması
elitepvpers — [Release] Auto Hunt - Python only (Eylül 2025)	https://www.elitepvpers.com/forum/metin2-hacks-bots-cheats-exploits-macros/5318086	SetPixelPosition + self.Debug() deseninin güncel botlarda birebir kullanımı
elitepvpers — Global Multihack yayınları	https://www.elitepvpers.com/forum/metin2-hacks-bots-cheats-exploits-macros/1454073	chr.SetPixelPosition(±2000) ışınla deseninin eski nesil tool'larda kanıtı (dump Crap_Levelbot ile aynı)
elitepvpers — Character Movement with packet (2020)	https://www.elitepvpers.com/forum/metin2-private-server/4701964	Tuşsuz/paket tabanlı hareket tekniğinin topluluk tartışması
MemoryBreak — GLBot++ Python Scripting API	https://memorybreak.com/glbot-python-api.php	Ticari bot framework'lerinde SetPixelPosition benzeri ışınlama API'lerinin standart varlığı

Tablo 11: Bulgunun genellenebilirliğini doğrulayan dış kaynaklar.